ER Management Desktop App — Implementation Tasks

84 tasks

MEMBER 1(Alexia): SYSTEM ARCHITECT & FOUNDATIONS

Responsible for project setup, database infrastructure, shared utilities, navigation shell, and cross-cutting concerns.

#	Task Name	Description
1.1	**Create C# WinUI 3 Solution Structure**	Create the Visual Studio C# WinUI 3 (Windows App SDK) solution with folder structure: Models, ViewModels, Views, Services, Repositories, DataAccess, Helpers. Add the Microsoft.Data.SqlClient NuGet package for local SQL Server connectivity (no ORMs). Also add CommunityToolkit.Mvvm NuGet package for MVVM support (ObservableObject, RelayCommand).
1.2	**Design SQL Server Database Schema**	Write a hand-written SQL DDL script to create all tables on the local SQL Server: Patient, ER_Visit, Triage, Triage_Parameters, ER_Room, Examination, Transfer_Log. Define primary keys, foreign keys, data types, and constraints matching the class diagram (no stored procedures)
1.3	**Seed Database with Initial Data**	Write a hand-written SQL INSERT script to populate the local SQL Server with sample data: at least 10 patients, 15 visits across various statuses, 8 rooms of different types (Operating Room, Trauma Bay, Respiratory Room, Neurology Room, Orthopedic Room, General Examination Room), and sample triage records.
1.4	**Implement DatabaseConnection Class**	Implement the DatabaseConnection class using Microsoft.Data.SqlClient (SqlConnection). Include a ConnectionString property read from appsettings.json, and Open()/Close() methods with proper connection lifecycle management.
1.5	**Implement SqlHelper Class**	Implement the SqlHelper utility with ExecuteReader() and ExecuteNonQuery() methods. These accept raw SQL strings and SqlParameter arrays, use DatabaseConnection for connection management, and handle proper disposal of connections, commands, and readers. All imports must reference Microsoft.Data.SqlClient instead of System.Data.SqlClient.
1.6	**Implement BaseViewModel Abstract Class**	Create the abstract BaseViewModel class by extending CommunityToolkit.Mvvm's ObservableObject (which implements INotifyPropertyChanged). Declare the IsBusy property using the [ObservableProperty] attribute. This is the base class all other ViewModels will inherit from — no manual INotifyPropertyChanged implementation required.

1.7	**Implement MainWindowViewModel**	Build MainWindowViewModel extending BaseViewModel with a INavigationService service property and navigation commands: ShowPatientRegistration(), ShowQueue(), ShowTriage(), ShowRoomAssignment(), ShowExamination(), ShowTransferLog(), ShowRoomManagement(). Use [RelayCommand] from CommunityToolkit.Mvvm instead of manually implementing ICommand. Each command instantiates and sets the corresponding ViewModel.
1.8	**Design MainWindow XAML Layout**	Create MainWindow.xaml using WinUI 3 controls. Use a NavigationView with MenuItems for each module as the sidebar, and a Frame as the main content area. Apply consistent WinUI 3 Fluent Design theming (RequestedTheme, MicaBackdrop or AcrylicBrush) and consistent fonts and spacing across the navigation shell.
1.9	**Implement View Navigation with DataTemplates**	In WinUI 3, navigation is done via Frame.Navigate(typeof(SomePage), viewModel) rather than DataTemplates. In the NavigationView.SelectionChanged handler, call contentFrame.Navigate() with the appropriate Page type and pass the ViewModel as a navigation parameter. Each View (Page) retrieves its ViewModel from the navigation parameter in its OnNavigatedTo() override. Verify that each module renders correctly on navigation.
1.10	**Implement Logging Utility**	Create a lightweight logging utility class that writes timestamped entries to a local text file. Support log levels (INFO, WARNING, ERROR). This utility will be used across all layers (Services, Repositories) for debugging and operational logging during development.
1.11	**Create Global Error Handling**	Implement a global exception handler via `Application.DispatcherUnhandledException` in `App.xaml.cs`. Log errors to a local text file and display user-friendly MessageBox dialogs without crashing the application.
1.12	**Set Up Dependency Injection**	Configure Microsoft.Extensions.DependencyInjection in App.xaml.cs. Register DatabaseConnection, SqlHelper, all Repositories, Services, and ViewModels with appropriate lifetimes (singleton for DB, transient for ViewModels). In WinUI 3, build the DI container in the App constructor and expose it via a static App.Services property, since there is no built-in generic host by default.
1.13	**Create App-Wide Styles and ResourceDictionary**	Define a ResourceDictionary with reusable WinUI 3 styles for Button, TextBox, ListView/DataGrid, TextBlock, ComboBox. Use WinUI 3's ThemeResource system and Fluent Design tokens for colors, corner radii, and spacing. Merge the dictionary in App.xaml to make styles available across all Pages.
1.14	**Implement App Configuration Management**	Use an appsettings.json file bundled with the app (set “Build Action” = Content in the project). Read the SQL Server connection string using Microsoft.Extensions.Configuration or a simple JsonDocument parser. Document the required SQL Server setup steps (instance name, database creation) for all team members.

MEMBER 2 (Miruna): PATIENT & VISIT MANAGEMENT — Feature 4: ER Visit State Management + CRUD

Responsible for patient registration, ER visit lifecycle, state management, and the patient/visit data layer.

ER Visit States: *`REGISTERED` → `TRIAGED` → `WAITING_FOR_ROOM` → `IN_ROOM` → `WAITING_FOR_DOCTOR` → `IN_EXAMINATION` → `TRANSFERRED` or `CLOSED`.*

Also valid: `TRIAGED` → `CLOSED`. Any invalid transition is rejected.

#	Task Name	Description
2.1	**Implement Patient Model**	Create the `Patient` class with properties: `Patient_ID`, `First_Name`, `Last_Name`, `Date_of_Birth`, `Gender`, `Phone`,
		`Emergency_Contact`, `Transferred`. Add C# validation methods for required fields. This supports the **CRUD operation**: Registration of new patient.
2.2	**Implement ER_Visit Model with State Field**	Create the `ER_Visit` class with properties: `Visit_ID`, `Patient_ID`, `Arrival_date_time`, `Chief_complaint`, `Status`. The `Status` field holds one of the 8 defined workflow states: REGISTERED, TRIAGED, WAITING_FOR_ROOM, IN_ROOM, WAITING_FOR_DOCTOR, IN_EXAMINATION, TRANSFERRED, CLOSED. This supports the **CRUD operation**: Create new ER visit.
2.3	**Implement ER_Visit State Transition Map**	Inside the `ER_Visit` model, define a C# dictionary that maps each state to its allowed next states: REGISTERED→{TRIAGED}, TRIAGED→{WAITING_FOR_ROOM, CLOSED}, WAITING_FOR_ROOM→{IN_ROOM}, IN_ROOM→{WAITING_FOR_DOCTOR}, WAITING_FOR_DOCTOR→{IN_EXAMINATION}, IN_EXAMINATION→{TRANSFERRED, CLOSED}.
2.4	**Implement CanTransitionTo and ChangeStatus**	Implement `ER_Visit.CanTransitionTo(newStatus)` which checks the transition map and returns true/false. Implement `ER_Visit.ChangeStatus(newStatus)` which calls `CanTransitionTo()` first and throws an `InvalidOperationException` with a descriptive message if the transition is invalid. Any invalid transition attempt is rejected.
2.5	**Implement StateManagementService**	Build `StateManagementService` with `ValidateTransition(currentStatus, newStatus)` and `ChangeVisitStatus(visitId, newStatus)`. This service loads the visit from the repository, calls `ER_Visit.ChangeStatus()`, and persists the new status via a hand-written UPDATE query. This supports the **CRUD operation**: Update ER visit status. All transition logic is in C#, not SQL.
2.6	**Implement PatientRepository**	Build `PatientRepository` using `SqlHelper` with hand-written SQL. `Add()` executes a parameterized INSERT INTO Patient query. `GetById()` executes a SELECT and manually maps `SqlDataReader` columns to a `Patient` object. No ORM mapping.

2.7	**Implement ERVisitRepository**	Build `ERVisitRepository` using `SqlHelper` with hand-written SQL. `Add()` inserts a new visit row with status REGISTERED. `GetActiveVisits()` selects all visits WHERE status NOT IN ('TRANSFERRED','CLOSED'). `UpdateStatus()` runs a parameterized UPDATE. `GetByVisitId()` retrieves a single visit. All queries use `SqlParameter`.
2.8	**Implement RegistrationService**	Create `RegistrationService` with `RegisterPatientAndVisit()`. In C#, create a `Patient` and `ER_Visit` object, validate the data, then call `PatientRepository.Add()` and `ERVisitRepository.Add()`. Set initial status to REGISTERED and record `Arrival_date_time`.
2.9	**Implement PatientRegistrationViewModel**	Build PatientRegistrationViewModel (extending BaseViewModel) with bindable properties: FirstName, LastName, DateOfBirth, Gender, Phone, EmergencyContact, ChiefComplaint — using [ObservableProperty] from CommunityToolkit.Mvvm. Implement RegisterPatientAndVisit() and ClearForm() using [RelayCommand]. Call RegistrationService from the command
2.10	**Design PatientRegistrationView XAML**	Create PatientRegistrationView.xaml as a WinUI 3 Page with labeled TextBoxes, a DatePicker, a ComboBox for Gender, and 'Register'/'Clear' buttons. Bind all controls to PatientRegistrationViewModel properties and commands using x:Bind (preferred in WinUI 3) or {Binding}.
2.11	**Add Input Validation and Feedback**	Implement validation in PatientRegistrationViewModel: require FirstName, LastName, DateOfBirth, ChiefComplaint. Validate phone format in C#. Show inline error messages and disable the Register button until valid. After success, show a WinUI 3 ContentDialog (not MessageBox, which does not exist in WinUI 3) with Patient ID, Visit ID, and status (REGISTERED). Auto-clear on success.
2.12	**Implement Visit Status History Tracking**	Hook into `StateManagementService.ChangeVisitStatus()` to persist every status change via `ERVisitRepository.UpdateStatus()`. Ensure `ER_Visit.CanTransitionTo()` is always called before any persist to validate the transition. Invalid transitions must throw a descriptive `InvalidOperationException`. All logic in C#, no new tables or repositories required.
2.13	**Implement Visit Details Panel**	Create a reusable UserControl that displays full visit details: patient info, arrival time, chief complaint, current status (one of the 8 workflow states), and triage level if applicable. Use hand-written SELECT with JOIN to fetch combined data from Patient and ER_Visit tables. Include state history timeline.
2.14	**Implement Visit Close Workflow**	Add a 'Close Visit' button. In C#, validate the visit is in an allowed state for closing (IN_EXAMINATION or TRIAGED) via StateManagementService.ChangeVisitStatus(). Transition to CLOSED. Persist via hand-written UPDATE and show a WinUI 3 ContentDialog confirmation (not MessageBox).

MEMBER 3 (Mihai): TRIAGE & QUEUE LOGIC — Features 1, 2, 3

Responsible for triage severity calculation, nurse request management, ER queue management, and related data layers.

Feature 1 *— ER Queue: ordered by triage severity (descending), then arrival time (ascending).*

Feature 2 *— Nurse Request: mock call to Staff Management to get an available nurse by* `availability_status = true`*.*

Feature 3 *— Triage Severity: critical condition auto-detection + weighted average formula.*

#	Task Name	Description
3.1	**Implement Triage Model**	Create the `Triage` class with properties: `Triage_ID`, `Visit_ID`, `Triage_Level`, `Specialization`, `Nurse_ID`, `Triage_Time`. This supports the **CRUD operation**: Create triage record.
3.2	**Implement Triage_Parameters Model**	Create `Triage_Parameters` with properties: `Triage_ID`, `Consciousness` (1–3), `Breathing` (1–3), `Bleeding` (1–3), `Injury_Type` (1–3), `Pain_Level` (1–3). Add C# validation to ensure each parameter is within range 1–3. Values: 3 = most critical, 1 = normal.
3.3	**Implement TriageRepository**	Build `TriageRepository` using `SqlHelper` with hand-written SQL. `Add()` executes a parameterized INSERT into the Triage table. `GetByVisitId()` executes a SELECT WHERE Visit_ID = @visitId and manually maps `SqlDataReader` to a `Triage` object.
3.4	**Implement TriageParametersRepository**	Build `TriageParametersRepository` with hand-written SQL. `Add()` inserts all five parameter values. `GetByTriageId()` selects parameters for a given Triage_ID. Map all columns manually from `SqlDataReader`.
3.5	**Implement Critical Condition Detection (Feature 3)**	In `Triage.CalculateTriageLevel()`, first check for critical conditions: if `consciousness = 3` OR `breathing = 3` OR `injury_type = 3` OR `bleeding = 3`, automatically assign **top priority** (triage level = 1). This is the first step of the triage algorithm, implemented entirely in C#.
3.6	**Implement Weighted Severity Score (Feature 3)**	If no critical condition is detected, calculate the weighted severity score: `severity_score = (consciousness × 3) + (breathing × 3) + (bleeding × 2) + (injury_type × 2) + (pain_level × 1)`. Map the resulting score to triage levels 2–5. Document the score-to-level mapping in code comments.
3.7	**Implement Specialization Determination**	Implement `Triage.DetermineSpecialization()` in C# using the following rules: `injury_type = 2` → Orthopedics; `breathing = 2` → Pulmonology; `consciousness = 2 OR 3` → Neurology; `bleeding = 3 OR injury_type = 3` → General Surgery; else → Emergency Medicine. This specialization is stored in the Triage record and used later by Room Assignment and Doctor Request.

3.8	**Implement Mock NurseService (Feature 2)**	Create a mock `NurseService` class simulating the external Staff Management system. Implement `RequestAvailableNurse()` which searches for a nurse where `availability_status = true` and returns a hardcoded `Nurse_ID`. Based on the external team's shared documentation.
3.9	**Implement TriageService**	Include `CalculateTriageLevel()` and `DetermineSpecialization()` method stubs Create `TriageService` with `CreateTriage()`, `CreateTriageParameters()`, and `RequestAvailableNurse()`. `CreateTriage()` calls `RequestAvailableNurse()` from the mock service, then `CalculateTriageLevel()` and `DetermineSpecialization()`, persists via repositories, and
		transitions the visit from REGISTERED → TRIAGED → WAITING_FOR_ROOM via `StateManagementService`.
3.10	**Implement TriageViewModel**	Build TriageViewModel (extending BaseViewModel) with properties: SelectedVisit, Consciousness, Breathing, Bleeding, InjuryType, PainLevel — using [ObservableProperty]. Implement commands LoadVisitsForTriage(), RequestAvailableNurse(), SaveTriage() using [RelayCommand]. After saving, trigger state transition to TRIAGED then WAITING_FOR_ROOM.
3.11	**Design TriageView XAML**	Create TriageView.xaml as a WinUI 3 Page. Use a DataGrid (from CommunityToolkit.WinUI.UI.Controls NuGet package) listing visits in REGISTERED status, ComboBoxes for each of the 5 parameters (e.g., Consciousness: 1-Awake, 2-Confused, 3-Unconscious), a 'Request Nurse' button, and a 'Save Triage' button. Show calculated severity and specialization. Use x:Bind to bind to TriageViewModel.
3.12	**Implement QueueService (Feature 1)**	Build `QueueService` with `GetOrderedQueue()`. Fetch all active visits and their triage data via hand-written SELECT with JOIN. Implement `OrderByTriageLevelAndArrivalTime()` in C#: sort by triage level ascending (level 1 = highest priority), then by `Arrival_date_time` ascending (earlier arrivals first).
3.13	**Implement QueueViewModel and Design QueueView**	Create QueueViewModel (extending BaseViewModel) with ActiveVisits list. Implement LoadQueue() and RefreshQueue() using [RelayCommand]. Design QueueView.xaml as a WinUI 3 Page with a DataGrid (from CommunityToolkit.WinUI.UI.Controls) showing: patient name, triage level, specialization, arrival time, and current status. Include a 'Refresh' button.
3.14	**Implement Triage Level Color Converter and Validation**	Create an IValueConverter in C# (namespace Microsoft.UI.Xaml.Data) mapping triage levels to colors (Level 1 = Red, 2 = Orange, 3 = Yellow, 4 = Green, 5 = Blue). Apply to QueueView DataGrid rows. Add triage validation: a visit must be selected, all 5 parameters filled (1–3), and the visit must be in REGISTERED status. Show error messages for failures.

MEMBER 4 (Andrei): DOCTOR REQUEST & EXAMINATION — Feature 5

Responsible for doctor request management, specialization-based doctor matching, examination workflow, and related data layers.

> ***Feature 5*** *— Doctor Request: after triage and room assignment, the system requests a doctor from Staff Management based on the required specialization.*
>
> *Specialization rules: `injury_type=2` → Orthopedics, `breathing=2` → Pulmonology, `consciousness=2|3` → Neurology, `bleeding=3|injury_type=3` → General Surgery, else → Emergency Medicine.*

#	Task Name	Description
4.1	**Implement Examination Model**	Create the `Examination` class with properties: `Exam_ID`, `Visit_ID`, `Doctor_ID`, `Exam_Time`, `Room_ID`, `Notes`. Add C# validation for required fields. This supports the **CRUD operation**: Create new examination record.
4.2	**Implement ExaminationRepository**	Build `ExaminationRepository` using `SqlHelper` with hand-written SQL. `Add()` executes a parameterized INSERT with all examination fields. `GetByVisitId()` executes a SELECT WHERE Visit_ID = @visitId. Manually map `SqlDataReader` to `Examination` objects. No ORM.
4.3	**Implement Mock StaffService for Doctor Request (Feature 5)**	Create a mock `StaffService` class simulating the external Staff Management system. Implement `RequestDoctor(string specialization, Triage_Parameters symptoms)` which receives the required specialization and patient symptoms, then returns a hardcoded doctor ID. Also implement `GetDoctorById(int id)` returning hardcoded doctor details (name, specialty).
4.4	**Implement ExaminationService**	Create `ExaminationService` with `RequestDoctor()` and `SaveExamination()`. `RequestDoctor()` retrieves the visit's triage specialization and symptoms, then calls the mock `StaffService.RequestDoctor(specialization, symptoms)`. `SaveExamination()` persists the examination via `ExaminationRepository`.
4.5	**Implement Doctor Request Workflow (Feature 5)**	When `RequestDoctor()` is called: (1) Read the triage record to get the specialization and symptoms, (2) send the specialization + symptoms to the mock Staff Management service, (3) receive the assigned `Doctor_ID`, (4) transition the visit from IN_ROOM to WAITING_FOR_DOCTOR via `StateManagementService`.
4.6	**Implement ExaminationViewModel**	Build ExaminationViewModel (extending BaseViewModel) with properties: SelectedVisit, DoctorId, Notes — using [ObservableProperty]. Implement RequestDoctor() and SaveExamination() using [RelayCommand]. Load visits in IN_ROOM status (ready for doctor request) and WAITING_FOR_DOCTOR status (ready for examination) via hand-written SELECT through ERVisitRepository.

4.7	**Design ExaminationView XAML**	Create ExaminationView.xaml as a WinUI 3 Page with a DataGrid (from CommunityToolkit.WinUI.UI.Controls) of visits in IN_ROOM and WAITING_FOR_DOCTOR status, a read-only field showing the assigned doctor's name and specialty, a multi-line TextBox for Notes, and 'Request Doctor' / 'Save Examination' buttons. Use x:Bind to bind to ExaminationViewModel.
4.8	**Implement Visit Status Updates for Examination**	When saving an examination, transition the visit from WAITING_FOR_DOCTOR to IN_EXAMINATION via StateManagementService.ChangeVisitStatus(). Show a WinUI 3 ContentDialog confirmation (not MessageBox) and refresh the list to remove the visit. The examination record stores Exam_Time, Doctor_ID, Room_ID, and Notes.
4.9	**Add Examination Validation Logic**	Validate in ExaminationViewModel (C# code): a visit must be selected, the visit must be in the correct status (IN_ROOM for doctor request, WAITING_FOR_DOCTOR for saving exam), a doctor must be assigned, and Notes must not be empty. Disable buttons until conditions are met. Show specific error messages via WinUI 3 ContentDialog or inline InfoBar controls.
4.10	**Display Examination History per Visit**	Add a section to `ExaminationView` showing past examinations for the selected visit. Fetch history via a hand-written SELECT in `ExaminationRepository.GetByVisitId()`. Display in a read-only DataGrid or ListBox below the main form.
4.11	**Add Doctor Assignment Display**	Enhance `ExaminationView` to display the assigned doctor's name and specialty (from the mock `StaffService.GetDoctorById()`) instead of just the doctor ID. Show the specialization that was used in the request alongside the assigned doctor info.
4.12	**Create Examination Summary Popup**	Build a WPF Window that combines patient info, triage results (severity level + specialization + all 5 parameters), and examination notes for a single visit. Fetch all data via hand-written SELECT with JOINs across Patient, ER_Visit, Triage, Triage_Parameters, and Examination tables. Display as read-only.
4.13	**Implement Examination Notes Auto-Save**	Add a timer-based auto-save feature for examination notes. Every 60 seconds, if notes have changed, persist them via a hand-written UPDATE query in `ExaminationRepository`. Show a subtle "Saved" indicator in the UI. Implement entirely in C#.
4.14	**Add Triage Details Display in Examination**	When a visit is selected in the ExaminationView, display a read-only panel showing its triage details: all 5 parameter values with their labels (e.g., Consciousness: Confused), calculated severity score, triage level, specialization, and assigned nurse ID. Fetch via hand-written SELECT with JOIN on Triage and Triage_Parameters.

MEMBER 5 (Alex): ROOM & RESOURCE MANAGEMENT — Features 6, 7

Responsible for automatic room assignment, room availability management, and the room data layer.

Feature 6 *— Automatic Room Assignment: select highest-priority visit from queue → determine room type based on specialization/symptoms → find matching available room → assign.*

Feature 7 *— Room Availability: rooms cycle through `available` → `occupied` → `cleaning` → `available`.*

#	Task Name	Description
5.1	**Implement ER_Room Model**	Create the `ER_Room` class with properties: `Room_ID`, `Room_Type`, `Availability_Status`. Implement `UpdateAvailabilityStatus(newStatus)` with C# validation enforcing allowed statuses: "available", "occupied", "cleaning". This supports the **CRUD operation**: Store rooms information.
5.2	**Implement Room Status Transition Validation (Feature 7)**	In `ER_Room.UpdateAvailabilityStatus()`, enforce C# logic for valid transitions following the Feature 7 workflow: `available` → `occupied` (patient enters), `occupied` → `cleaning` (patient leaves), `cleaning` → `available` (after cleaning). Reject invalid transitions with descriptive error messages. This supports the **CRUD operation**: Update room availability.
5.3	**Implement RoomRepository**	Build `RoomRepository` using `SqlHelper` with hand-written SQL. `GetAvailableRooms()` runs `SELECT ... WHERE Availability_Status = 'available'`. `GetById()` selects by Room_ID. `UpdateAvailabilityStatus()` runs a parameterized UPDATE. `GetRoomsByStatus(status)` returns rooms filtered by any status. `GetAllRooms()` returns all rooms. Map results manually from `SqlDataReader`.
5.4	**Implement Room Type Determination Algorithm (Feature 6)**	Implement the room type determination algorithm from Feature 6 in C#: IF specialization == "General Surgery" OR bleeding == 3 OR injury_type == 3 → "Operating Room (OR)"; ELSE IF consciousness == 3 OR breathing == 3 → "Trauma/Resuscitation Bay"; ELSE IF specialization == "Pulmonology" OR breathing == 2 → "Respiratory/Monitored Room"; ELSE IF specialization == "Neurology" OR consciousness == 2 → "Neurology/Quiet Observation Room"; ELSE IF specialization == "Orthopedics" OR injury_type == 2 → "Orthopedic/Procedure Room"; ELSE → "General Examination Room".
5.5	**Implement RoomAssignmentService (Feature 6)**	Create `RoomAssignmentService` with `FindAvailableRoom()`, `AssignRoomToVisit()`, `UpdateRoomAvailability()`. `FindAvailableRoom()` determines the required room type using the algorithm from 5.4, then searches for rooms with status "available" that match the type. `AssignRoomToVisit()` assigns the room, marks it as "occupied", and transitions the visit from WAITING_FOR_ROOM to IN_ROOM via `StateManagementService`.

5.6	**Implement Automatic Room Assignment Logic (Feature 6)**	Implement the full automatic assignment flow: (1) select the highest-priority visit from the queue (highest triage severity, earliest arrival), (2) determine required room type, (3) search for matching available rooms, (4) assign the first matching room. If no suitable room is available, the patient remains in WAITING_FOR_ROOM.
5.7	**Implement RoomAssignmentViewModel**	Build RoomAssignmentViewModel (extending BaseViewModel) with WaitingVisits and AvailableRooms as ObservableCollection properties using [ObservableProperty]. Implement LoadData(), AssignRoom(), and ManualAssignRoom() using [RelayCommand]. LoadData() fetches both lists via repositories.
5.8	**Design RoomAssignmentView XAML**	Create RoomAssignmentView.xaml as a WinUI 3 Page with two DataGrids (from CommunityToolkit.WinUI.UI.Controls) side-by-side: visits in WAITING_FOR_ROOM status and available rooms. Add 'Auto-Assign Room' and 'Manual Assign' buttons. Show a WinUI 3 ContentDialog confirmation (not MessageBox) after assignment and refresh both lists.
5.9	**Add Room Assignment Validation**	Validate in RoomAssignmentViewModel (C# logic): for manual assignment, both a visit and room must be selected, the room must have status 'available', and the visit must be in WAITING_FOR_ROOM. For auto-assignment, check that there are waiting visits. Show specific error messages via WinUI 3 ContentDialog for each invalid case.
5.10	**Implement RoomManagementService (Feature 7)**	Build `RoomManagementService` with `GetAvailableRooms()`, `GetOccupiedRooms()`, `GetCleaningRooms()`, and `MarkRoomAsCleaned()`. Each getter calls `RoomRepository` with a hand-written SELECT filtered by status. `MarkRoomAsCleaned()` validates the transition (cleaning → available) in C# then updates via repository.
5.11	**Implement RoomManagementViewModel**	Create RoomManagementViewModel (extending BaseViewModel) with three ObservableCollection<ER_Room> properties: AvailableRooms, OccupiedRooms, CleaningRooms — using [ObservableProperty]. Implement LoadRooms() and MarkRoomAsCleaned() using [RelayCommand].
5.12	**Design RoomManagementView XAML**	Create RoomManagementView.xaml as a WinUI 3 Page with three sections or a TabView (WinUI 3 TabView control): Available Rooms, Occupied Rooms, and Cleaning Rooms. Add a 'Mark Room as Cleaned' button in the Cleaning section. Use x:Bind to bind all sections to RoomManagementViewModel.
5.13	**Auto-Set Room to Cleaning on Visit End (Feature 7)**	When a visit reaches TRANSFERRED or CLOSED, automatically set the associated room to "cleaning". Hook into `StateManagementService.ChangeVisitStatus()` — when status becomes TRANSFERRED or CLOSED, look up the room from the Examination table (hand-written SELECT) and call `UpdateAvailabilityStatus("cleaning")`.

5.14	**Implement Room Capacity Dashboard and Color Coding**	Add a summary panel to RoomManagementView showing total rooms, available/occupied/cleaning counts. Create an IValueConverter in C# (namespace Microsoft.UI.Xaml.Data) mapping room types to colors (e.g., Operating Room = Red, Trauma Bay = Orange, General = Green). Apply to both RoomAssignment and RoomManagement DataGrids. Calculate stats in C#.

MEMBER 6 (Gabi): EXTERNAL INTEGRATION & AUDITING — Features 8, 9

Responsible for data transfer to Patient Management system, transfer logging, and audit features.

> ***Feature 8*** *— Data Transfer: send patient data (CNP, name, DOB, gender, phone, emergency contact, visit ID, triage symptoms, exam date) to Patient Management system via simulated API call.*
>
> ***Feature 9*** *— Transfer Logging: each transfer attempt is logged with status: SUCCESS, FAILED, or RETRYING.*

#	Task Name	Description
6.1	**Implement Transfer_Log Model (Feature 9)**	Create the `Transfer_Log` class with properties: `Transfer_ID`, `Visit_ID`, `Transfer_Time`, `Target_system`, `Status` (string: "SUCCESS", "FAILED", or "RETRYING"). Add C# validation: `Target_system` must not be empty, `Visit_ID` must be valid, `Status` must be one of the three allowed values.
6.2	**Implement TransferLogRepository**	Build `TransferLogRepository` using `SqlHelper` with hand-written SQL. `Add()` executes a parameterized INSERT with all transfer log fields. `GetByVisitId()` executes a SELECT WHERE Visit_ID = @visitId. `GetAll()` fetches all transfer logs. Map `SqlDataReader` results to `Transfer_Log` objects manually. No ORM.
6.3	**Implement Patient Data Package (Feature 8)**	Create a data transfer object (DTO) or method that packages the required patient data for transfer: Patient identifier (CNP), First Name, Last Name, Date of Birth, Gender, Phone, Emergency Contact, ER visit identifier, triage results (all symptoms/parameters), and examination date. Fetch all data via hand-written SELECTs with JOINs across Patient, ER_Visit, Triage, Triage_Parameters, and Examination tables.
6.4	**Implement TransferService (Feature 8)**	Create `TransferService` with `SendPatientData()` and `LogTransfer()`. `SendPatientData()` collects the patient data package defined in Feature 8 and simulates an API call to the Patient Management system. Since there's no server, save the serialized data (JSON) to a local file. `LogTransfer()` persists the transfer record via `TransferLogRepository`.
6.5	**Implement Local File-Based Transfer Simulation (Feature 8)**	Simulate the API call to the external Patient Management system by serializing the patient data package to a JSON file saved in a configurable local directory. Store the file path in the `Transfer_Log` table. If another user logs in, they can see incoming transfers by reading the file from the stored path on the local disk.
6.6	**Implement Transfer Logging (Feature 9)**	After each transfer attempt, create a `Transfer_Log` entry with: `Transfer_ID`, `Visit_ID`, current timestamp, target system name ("Patient Management"), and status. Set status to "SUCCESS" if file was saved correctly, "FAILED" if an error occurred, or "RETRYING" if a retry is in progress. Each attempt creates a separate log entry.
6.7	**Implement TransferLogViewModel**	Build TransferLogViewModel (extending BaseViewModel) with SelectedVisit property using [ObservableProperty]. Implement SendPatientData() and LoadLogs() using [RelayCommand]. Load only visits in IN_EXAMINATION status as eligible for transfer. Show transfer log entries for the selected visit.
6.8	**Design TransferLogView XAML**	Create TransferLogView.xaml as a WinUI 3 Page with a DataGrid (from CommunityToolkit.WinUI.UI.Controls) of visits eligible for transfer (status = IN_EXAMINATION), a 'Send Patient Data' button, and a history DataGrid showing past transfers: Transfer_ID, timestamp, target system, status. Use x:Bind to bind to TransferLogViewModel.

6.9	**Add Transfer Validation Logic**	Validate in TransferLogViewModel (C# logic): a visit must be selected, the visit must be in IN_EXAMINATION status, and the patient must not already have a successful transfer (Transferred = true). Show specific error messages via WinUI 3 ContentDialog for each validation failure.
6.10	**Update Visit & Patient Status After Transfer (Feature 8)**	After a successful transfer (status = 'SUCCESS'), call StateManagementService.ChangeVisitStatus() to transition the visit from IN_EXAMINATION to TRANSFERRED. Update Patient.Transferred to true via a hand-written UPDATE query in PatientRepository. Show a WinUI 3 ContentDialog success message and refresh the view.
6.11	**Implement Transfer Retry Mechanism (Feature 9)**	If `SendPatientData()` fails, set the log status to "FAILED" and enable a "Retry" button. When retrying, create a new log entry with status "RETRYING". On success, update the entry to "SUCCESS". Log each attempt separately with timestamps. Display all attempts in the transfer history DataGrid.
6.12	**Implement TransferService GetLogs**	Implement `TransferService.GetLogs(visitId)` to support `TransferLogViewModel.LoadLogs()`. The method calls `TransferLogRepository.GetByVisitId()` using a hand-written SELECT and returns a list of `Transfer_Log` objects mapped manually from SqlDataReader. This completes the read path of the transfer log as defined in the class diagram.
6.13	**Wire TransferLogViewModel to Display Transfer History**	In `TransferLogViewModel.LoadLogs()`, call `TransferService.GetLogs(visitId)` and bind the results to the transfer history DataGrid in `TransferLogView.xaml`. Display all `Transfer_Log` entries for the selected visit, showing Transfer_ID, timestamp, target system, and status. Refresh the log list after every `SendPatientData()` call.
6.14	**Display Transfer Status in TransferLogView**	In TransferLogView.xaml, display transfer status clearly for each log entry using color coding: SUCCESS in green, FAILED in red, RETRYING in orange. Implement the color mapping as an IValueConverter in C# (namespace Microsoft.UI.Xaml.Data). Each row in the transfer history DataGrid must show Transfer_ID, timestamp, target system, and status as defined in Feature 9.